

8. Code Architecture / Folder Structure

Purpose

This document defines how the source code of the project is organized.

The goal is to keep the project:

- simple
- modular
- easy to navigate
- easy to extend later

The system is divided into three main code areas:

1. frontend
2. backend
3. planner

Root Project Structure

```
project-root/  
|  
├ docs/  
├ frontend/  
├ backend/  
└ planner/
```

docs

Contains project documentation, architecture notes, decisions, and development plan.

frontend

Contains the web user interface and map visualization.

backend

Contains the FastAPI application and API endpoints.

planner

Contains route planning logic and obstacle-processing logic.

Frontend Structure

```
frontend/
├── src/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── state/
│   ├── types/
│   ├── utils/
│   └── App.jsx / App.tsx
├── public/
└── package.json
```

components

Reusable UI pieces.

Examples:

- MapView
- ControlPanel
- InfoPanel
- DroneMarker
- ObstacleLayer
- RouteLayer

pages

Top-level pages or screens.

For this project, you may only need:

- MainPage

services

Functions for communicating with backend.

Examples:

- routeApi
- missionApi
- obstacleApi

state

Stores frontend application state.

Examples:

- selected drone
- start point
- goal point
- obstacle list
- route
- drone status

types

Shared frontend data models.

Examples:

- Drone
- Mission
- Route
- Obstacle

utils

Helper functions.

Examples:

- coordinate conversion
 - route formatting
 - map helpers
-

Backend Structure

```
backend/
|
├─ app/
|   │
|   ├─ api/
|   │
|   ├─ models/
|   │
|   ├─ schemas/
|   │
|   ├─ services/
|   │
|   ├─ core/
|   │
|   └─ main.py
|
└─ requirements.txt
```

api

Contains FastAPI route definitions.

Examples:

- route_endpoints.py
- mission_endpoints.py
- obstacle_endpoints.py

models

Internal backend models or domain models.

Examples:

- drone.py
- mission.py
- obstacle.py

- route.py

schemas

Request and response schemas for FastAPI.

Examples:

- route_request.py
- route_response.py
- mission_command.py

services

Business logic layer.

Examples:

- route_service.py
- mission_service.py
- obstacle_service.py

core

Shared backend utilities and configuration.

Examples:

- config.py
- constants.py
- overpass_client.py

main.py

Application entry point.

Planner Structure

planner/

```
|
| algorithms/
| grid/
| geometry/
| models/
| planner.py
```

algorithms

Contains route generation logic.

Examples:

- astar.py
- smoothing.py

grid

Contains code for internal planning representation.

Examples:

- occupancy_grid.py
- grid_builder.py

geometry

Contains geometry processing.

Examples:

- polygon_blocking.py
- obstacle_conversion.py
- height_rules.py

models

Planner-specific data models.

Examples:

- node.py

- `planner_obstacle.py`
- `planner_route.py`

planner.py

Main planner interface used by backend.

This file should expose something like:

```
generate_route(start, goal, obstacles, drone_params)
```

Dependency Direction

The system should follow this dependency order:

Frontend → Backend API → Planner

Important rule:

- frontend must not directly contain route algorithm logic
 - backend must not contain large algorithm code inside API files
 - planner must not depend on frontend
-

Responsibility Separation

Frontend responsibilities

- show map
- collect user input
- display obstacles
- display route
- animate drone marker
- show height and mission status

Backend responsibilities

- accept requests
- validate data
- retrieve map obstacles
- call planner
- return response

Planner responsibilities

- transform obstacles into planning model
 - compute route
 - apply path improvements
 - handle height constraints
-

Multi-Drone Readiness

Even though version 1 supports one drone, the code should be prepared for multiple drones.

This means:

- use `drone_id` in route and mission models
 - store drone state in reusable structures
 - avoid hardcoding a single drone everywhere
-

General Rule

Keep each module focused:

- API files only define endpoints
- service files handle logic
- planner files handle algorithms

- UI components remain small and reusable

This separation is important for both development and thesis documentation.